Instituto tecnológico de Costa Rica Centro Académico Limón

Tarea #1 gramática BNF

Estudiantes:

Elder Leon Pérez 2023166120

Quiriat Mata Araya 2023379891

Profesor:

Allan Rodríguez Davila

Fecha entrega: 28/03/2026

Repositorio = https://github.com/Q-ARAYA/Gramatica-BNF-Tarea-1-Compiladores

Contents

Nota: 4
Descripción del problema 4
Diseño de programa 4
Símbolo inicial 4
No terminales 4
Léxicos base 4
Literales y tipos 5
Identificadores 5
Delimitadores y operadores 5
Comentarios 5
Expresiones aritméticas 6
Expresiones relacionales 6
Expresiones lógicas 6
Expresiones generales y llamadas 6
Variables 7
Arreglos 7
Entrada y salida 7
Estructuras de control 7
Sentencias y bloques 8
Funciones y programa 8
Terminales 8
Dígitos 8
Letras 8
Delimitadores y operadores de bloque 8
Comentarios 9
Operadores aritméticos 9

Operadores lógicos 9
Operadores relacionales 9
Palabras reservadas 9
Producciones 9
Léxicos base 9
Literales y tipos 10
Identificadores 10
Delimitadores y operadores 10
Comentarios 10
Expresiones aritméticas 11
Expresiones relacionales 11
Expresiones lógicas 11
Expresiones generales y llamadas 12
Variables 12
Arreglos 12
Entrada y salida 12
Estructuras de control 13
Sentencias y bloques 13
Funciones y programa 13
Lecciones aprendidas 14
Objetivos alcanzados 14
Objetivos no logrados 14
Ejecuciones de ejemplo 15

Nota:

El repositorio encontrara 3 archivos este documento además de una carpeta que contiene un archivo. bnf extensión de vs únicamente para que se vea bonito y ordenado o un txt con todos los terminales no terminales y producciones.

Extensión: https://marketplace.visualstudio.com/items?itemName=Vallentin.vscode-bnf

Descripción del problema

El trabajo consiste en crear desde cero una gramática para un compilador. En esta primera tarea se pide seguir los requerimientos dados y usar la estructura que el profesor definió previamente. La idea es enfocarse solo en la parte léxica y sintáctica del lenguaje, sin tomar en cuenta el análisis semántico.

Diseño de programa

Símbolo inicial

<programa>

No terminales

Léxicos base

- <letra>
- <digito>
- <digitoSincero>
- <lista_digitos>
- <texto>

Literales y tipos

- <entero>
- <flotante>
- <exponencial>

- <fraccion>
- <bool>
- <cadena>
- <caracter>
- <elemento_numerico>
- <elemento>
- <tipo>
- <tipo_funcion>

Identificadores

- <ID>
- <resto_id>

Delimitadores y operadores

- <abrir_bloque>
- <cerrar_bloque>
- <fin_sentencia>
- <asignacion>
- <separador>
- <coma>
- <parentesis_apertura>
- <parentesis_cierre>
- <abrirParentesis_cuadrado>
- <cerrarParentesis_cuadrado>

Comentarios

- <comentario_inicio_linea>
- <comentario_inicio_bloque>
- <comentario_fin_bloque>
- <comentario_linea>
- <comentario_bloque>
- <comentario>
- <comentarioInicio>
- <comentariofinal>

Expresiones aritméticas

- <factor>
- <potencia>
- <resto_potencia>
- <termino>
- <resto_termino>
- <expresion_aritmetica>
- <resto_aritmetica>

Expresiones relacionales

- <operando_relacional_eq>
- <less_t>
- <less_te>
- <greather_t>
- <greather_te>
- <equal>
- <n_equal>
- <expresion_relacional>

Expresiones lógicas

- <factor_logico>
- <resto_termino_logico>
- <termino_logico>
- <resto_logico>
- <expresion_logica>

Expresiones generales y llamadas

- <expresion>
- <lista_expresiones>
- <llamada_funcion>

Variables

- <declarar_variable>
- <asignar_variable>
- <variable>

Arreglos

- <lista_enteros>
- <lista_flotantes>
- <fila_enteros>
- <fila_flotantes>
- <filas_enteros>
- <filas_flotantes>
- <vector_enteros>
- <vector_flotantes>
- <matriz_enteros>
- <matriz_flotantes>
- <declarar_arreglo_lista>
- <declarar_arreglo_matriz>
- <declarar_arreglo>
- <crear_arreglo_lista>
- <crear_arreglo_matriz>
- <crear_arreglo>
- <acceder_arreglo_lista>
- <acceder_arreglo_matriz>
- <acceder_arreglo>
- <modificar_arreglo_lista>
- <modificar_arreglo_matriz>
- <modificar_arreglo>
- <arreglo>

Entrada y salida

- <cin>
- <cout>
- <LecturaEscritura>

Estructuras de control

- <else>
- <if>
- <do-while>
- <cases>
- <case_item>
- <switch>
- <instrucion>

Sentencias y bloques

- <break>
- <return>
- <bloque>
- <sentencia>
- <lista_sentencias>
- <lista_elementos>

Funciones y programa

- <param>
- <lista_params>
- <funcion>
- <lista_funciones>
- <main>
- <programa>

Terminales

Dígitos

"0" "1" "2" "3" "4" "5" "6" "7" "8" "9"

Letras

"a" ... "z" "A" ... "Z"

Delimitadores y operadores de bloque

"|:" (inicio de bloque)

":|" (cierre de bloque)

"!" (fin de sentencia)

"<-" (asignacion)

"~" (separador)

"," (separador parametros)

"<|" (apertura parentesis)

"|>" (cierre parentesis)

"<<" (apertura indice)

">>" (cierre indice)

":" (dos puntos)

Comentarios

"¡¡" (comentario de linea)

"{-" (inicio comentario de bloque)

"-}" (cierre comentario de bloque)

Operadores aritméticos

- "+"
- "-"
- "*"
- "/"
- "%"
- "^"

Operadores lógicos

- "@" (and)
- "#" (or)
- "$" (not)

Operadores relacionales

- "equal"
- "n_equal"
- "less_t"
- "less_te"
- "greather_t"
- "greather_te"

Palabras reservadas

- "int"
- "bool"
- "float"
- "string"
- "char"
- "if"
- "else"
- "switch"
- "case"
- "default"
- "return"
- "break"
- "cin"
- "cout"
- "true"
- "false"
- "empty"
- "__main__"

Producciones

Léxicos base

<letra>::= [a-zA-Z]

<texto> ::= <letra><texto>| <digito><texto>| " " <texto> | ε

Literales y tipos

<digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<digitoSincero> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<lista_digitos> ::= <digito> <lista_digitos> | ε

<entero> ::= "0" | <digitoSincero> <lista_digitos>

<flotante> ::= <entero> "." <entero>

<exponencial> ::= <entero> "e" <entero>

<fraccion> ::= <entero> "/" <entero>

<bool> ::= "true" | "false"

<cadena> ::= "\"" <texto> "\""

<caracter> ::= "'" <letra> "'"

<elemento_numerico> ::= <entero> | <flotante> | <exponencial> | <fraccion>

<elemento> ::= <elemento_numerico> | <bool> | <cadena> | <caracter>

<tipo> ::= "int" | "bool" | "float" | "string" | "char"

<tipo_funcion> ::= "int" | "bool" | "float" | "char"

Identificadores

<ID> ::= <letra> <resto_id>

<resto_id> ::= <letra> <resto_id> | <digito> <resto_id> | ε

Delimitadores y operadores

<abrir_bloque> ::= "|:"

<cerrar_bloque> ::= ":|"

<fin_sentencia> ::= "!"

<asignacion> ::= "<-"

<separador> ::= "~"

<coma> ::= ","

<parentesis_apertura> ::= "<|"

<parentesis_cierre> ::= "|>"

<abrirParentesis_cuadrado> ::= "<<"

<cerrarParentesis_cuadrado> ::= ">>"

Comentarios

<comentario_inicio_linea> ::= "¡¡"

<comentario_inicio_bloque> ::= "{-"

<comentario_fin_bloque> ::= "-}"

<comentario_linea> ::= <comentario_inicio_linea> <texto>

<comentario_bloque> ::= <comentario_inicio_bloque> <texto> <comentario_fin_bloque>

<comentario> ::= <comentario_linea> | <comentario_bloque>

Expresiones aritméticas

<factor> ::= <elemento_numerico> | <ID> | <cadena> | <parentesis_apertura> <expresion> <parentesis_cierre> | <acceder_arreglo> | "++" <ID> | "--" <ID> | "-" <factor> | <llamada_funcion>

<potencia> ::= <factor> <resto_potencia>

<resto_potencia> ::= "^" <factor> <resto_potencia> | ε

<termino> ::= <potencia> <resto_termino>

<resto_termino> ::= "*" <potencia> <resto_termino>| "/" <potencia> <resto_termino>| "%" <potencia> <resto_termino>| ε

<expresion_aritmetica> ::= <termino> <resto_aritmetica>

<resto_aritmetica> ::= "+" <termino> <resto_aritmetica> | "-" <termino> <resto_aritmetica> | ε

Expresiones relacionales

<operando_relacional_eq> ::= <expresion_aritmetica>| <bool>| <ID>| <llamada_funcion>| <acceder_arreglo>

<less_t> ::= "less_t" <parentesis_apertura> <expresion_aritmetica> <coma> <expresion_aritmetica> <parentesis_cierre>

<less_te> ::= "less_te" <parentesis_apertura> <expresion_aritmetica> <coma> <expresion_aritmetica> <parentesis_cierre>

<greather_t> ::= "greather_t" <parentesis_apertura> <expresion_aritmetica> <coma> <expresion_aritmetica> <parentesis_cierre>

<greather_te> ::= "greather_te" <parentesis_apertura> <expresion_aritmetica> <coma> <expresion_aritmetica> <parentesis_cierre>

<equal> ::= "equal" <parentesis_apertura> <operando_relacional_eq> <coma> <operando_relacional_eq> <parentesis_cierre>

<n_equal> ::= "n_equal" <parentesis_apertura> <operando_relacional_eq> <coma> <operando_relacional_eq> <parentesis_cierre>

<expresion_relacional> ::= <equal> | <n_equal>| <less_t> | <less_te> | <greather_t> | <greather_te>

Expresiones lógicas

<factor_logico> ::= "$" <factor_logico> | <expresion_relacional> | <bool> | <parentesis_apertura> <expresion_logica> <parentesis_cierre>|<ID>|<llamada_funcion>|<acceder_arreglo>

<resto_termino_logico> ::= "@" <factor_logico> <resto_termino_logico> | ε

<termino_logico> ::= <factor_logico> <resto_termino_logico>

<resto_logico> ::= "#" <termino_logico> <resto_logico> | ε

<expresion_logica> ::= <termino_logico> <resto_logico>

Expresiones generales y llamadas

<lista_expresiones> ::= <expresion> | <expresion> <coma> <lista_expresiones> | ε

<llamada_funcion> ::= <ID> <parentesis_apertura> <lista_expresiones> <parentesis_cierre>

<expresion> ::= <expresion_logica> | <expresion_aritmetica>

Variables

<declarar_variable> ::= <tipo> <separador> <ID> | <tipo> <separador> <ID> <asignacion> <expresion>

<asignar_variable> ::= <ID> <asignacion> <expresion>

<variable>::= <declarar_variable> | <asignar_variable>

Arreglos

<lista_enteros> ::= <entero>| <entero> <coma> <lista_enteros>

<lista_flotantes> ::= <flotante> | <flotante> <coma> <lista_flotantes>

<fila_enteros> ::= <abrir_bloque> <lista_enteros> <cerrar_bloque>

<fila_flotantes> ::= <abrir_bloque> <lista_flotantes> <cerrar_bloque>

<filas_enteros> ::= <fila_enteros>| <fila_enteros> <coma> <filas_enteros>

<filas_flotantes> ::= <fila_flotantes>| <fila_flotantes> <coma> <filas_flotantes>

<vector_enteros> ::= <abrir_bloque> <lista_enteros> <cerrar_bloque>

<vector_flotantes> ::= <abrir_bloque> <lista_flotantes> <cerrar_bloque>

<matriz_enteros> ::= <abrir_bloque> <filas_enteros> <cerrar_bloque>

<matriz_flotantes> ::= <abrir_bloque> <filas_flotantes> <cerrar_bloque>

<declarar_arreglo_lista> ::= "int" <separador> <ID> <abrirParentesis_cuadrado> <entero> <cerrarParentesis_cuadrado>| "float" <separador> <ID> <abrirParentesis_cuadrado> <entero> <cerrarParentesis_cuadrado>

<declarar_arreglo_matriz> ::= "int" <separador> <ID> <abrirParentesis_cuadrado> <entero> <coma> <entero> <cerrarParentesis_cuadrado>| "float" <separador> <ID> <abrirParentesis_cuadrado> <entero> <coma> <entero> <cerrarParentesis_cuadrado>

<declarar_arreglo> ::= <declarar_arreglo_lista>| <declarar_arreglo_matriz>

<crear_arreglo_lista> ::= "int" <separador> <ID> <asignacion> <vector_enteros>| "float" <separador> <ID> <asignacion> <vector_flotantes>

<crear_arreglo_matriz> ::= "int" <separador> <ID> <asignacion> <matriz_enteros>| "float" <separador> <ID> <asignacion> <matriz_flotantes>

<crear_arreglo> ::= <crear_arreglo_matriz>| <crear_arreglo_lista>

Entrada y salida

<cin> ::= "cin" <separador> <ID> | "cin" <separador> <acceder_arreglo>

<cout> ::= "cout" <separador> <expresion>

<LecturaEscritura>::= <cin> <fin_sentencia>| <cout> <fin_sentencia>

Estructuras de control

<else> ::= "else" <bloque> | ε

<if>::= "if" <parentesis_apertura> <expresion_logica> <parentesis_cierre> <bloque> <else>

<do-while> ::= "do" <bloque> "while" <separador> <parentesis_apertura> <expresion_logica> <parentesis_cierre> <fin_sentencia>

<cases> ::= <case_item> <cases>| ε

<case_item> ::="case"<separador><elemento>":"<lista_sentencias>|"default" ":"<lista_sentencias>

<switch> ::= "switch" <parentesis_apertura> <expresion> <parentesis_cierre> <abrir_bloque> <cases> <cerrar_bloque>

<instrucion> ::= <if> | <do-while> | <switch>

Sentencias y bloques

<break> ::= "break"

<return> ::= "return" <separador> <expresion>

<lista_elementos> ::= <elemento> | <elemento> <coma> <lista_elementos> | ε

<bloque> ::= <abrir_bloque> <lista_sentencias> <cerrar_bloque>

<sentencia> ::= <variable><fin_sentencia>|<arreglo><fin_sentencia>| <instrucion>| <bloque> | <return> <fin_sentencia>| <break> <fin_sentencia> | <LecturaEscritura>| <expresion> <fin_sentencia>

<lista_sentencias> ::= <sentencia> <lista_sentencias> | <comentario> <lista_sentencias> | ε

Funciones y programa

<param> ::= <tipo> <separador> <ID>

<lista_params> ::= <param> | <param> <coma> <lista_params> | ε

<funcion> ::= <tipo_funcion> <separador> <ID> <parentesis_apertura> <lista_params> <parentesis_cierre> <bloque>

<lista_funciones> ::= <funcion> <lista_funciones> | <comentario> <lista_funciones> | ε

<main> ::= "empty"<separador>"__main__"<parentesis_apertura><parentesis_cierre><abrir_bloque><lista_sentencias><cerrar_bloque>

<programa> ::= <lista_funciones> <main>

Lecciones aprendidas

Una de las principales lecciones fue entender la diferencia entre análisis sintáctico y semántico. Se evidenció que una expresión puede ser válida para la gramática aunque no tenga sentido lógico, Además, se aprendió la importancia de definir correctamente los terminales y no terminales, ya que pequeños errores (como en palabras reservadas o

símbolos) provocan fallos desde el inicio del análisis. También se reforzó el uso de estructuras como bloques, listas de sentencias y expresiones recursivas.

Objetivos alcanzados

- Se logró diseñar una gramática que reconoce estructuras principales como:
    - Declaración de variables
    - Asignaciones
    - Lectura y escritura (cin, cout)
    - Llamadas a funciones
    - Estructuras de control como if-else
- Se respetaron los símbolos y formato definidos (uso de ~, <| |>, |: :|, etc.)
- Se consiguió validar correctamente strings de prueba que cumplen la estructura del lenguaje
- Se logró separar correctamente la lógica del lenguaje en niveles (expresiones, sentencias, bloques)

Objetivos no logrados

Se espera haber logrado cumplir con los objetivos propuestos en la construcción de la gramática. Sin embargo, es posible que existan errores, algunos de los cuales podrían corregirse en una etapa posterior mediante el análisis semántico, mientras que otros pueden deberse a detalles en la gramática desarrollada. De igual forma, se acepta y se espera la retroalimentación para mejorar y ajustar el trabajo.

Ejecuciones de ejemplo

BNFlab LOCAL

Cargar ejemplo | Limpiar | Compilar gramática

GRAMÁTICA BNF/EBNF — CTRL+ENTER PARA COMPILAR

```
<fin_sentencia>| <break> <fin_sentencia> |
<LecturaEscritura>| <expresion> <fin_sentencia>
<lista_sentencias> ::= <sentencia> <lista_sentencias> |
<comentario> <lista_sentencias> | ε
<LecturaEscritura>::= <cin> <fin_sentencia>| <cout>
<fin_sentencia>
<bloque> ::= <abrir_bloque> <lista_sentencias>
<cerrar_bloque>
<break> ::= "break"
<cin> ::= "cin" <separador> <ID> | "cin" <separador>
<acceder_arreglo>
```

REGLAS DETECTADAS — <programa>

<letra> <texto> <comentario_inicio_linea> <comentario_inicio_bloque> <comentario_fin_bloque> <comentario_linea> <comentario_bloque> <comentario>

TEXTO A VALIDAR — CTRL+ENTER PARA VALIDAR

```
empty~__main__◁▷|:int~x!int~y!int~resultado!cin~x!cin~y!r
esultado←x+y!cout~resultado!:|
```

▶ Validar entrada | ✦ Generar ejemplos

TERMINAL DE RESULTADOS — 169PX | Restablecer | Limpiar

Reglas: 105 Validaciones: 3 Correctas: 1 Fallidas: 2 Ejemplos: 0

✓ ✓ VÁLIDO CONTRA
La entrada `empty~__main__<||>|:int~x!int~y!int~resultado!cin~x!cin~y!resultado<-x+y!cout~resultado!:|` satisface la gramática.

GRAMÁTICA BNF/EBNF — CTRL+ENTER PARA COMPILAR

```
<fin_sentencia>| <break> <fin_sentencia> |
<LecturaEscritura>| <expresion> <fin_sentencia>
<lista_sentencias> ::= <sentencia> <lista_sentencias> |
<comentario> <lista_sentencias> | ε
<LecturaEscritura>::= <cin> <fin_sentencia>| <cout>
<fin_sentencia>
<bloque> ::= <abrir_bloque> <lista_sentencias>
<cerrar_bloque>
<break> ::= "break"
<cin> ::= "cin" <separador> <ID> | "cin" <separador>
<acceder_arreglo>
```

REGLAS DETECTADAS — <programa>

<letra> <texto> <comentario_inicio_linea> <comentario_inicio_bloque> <comentario_fin_bloque> <comentario_linea> <comentario_bloque> <comentario>

TEXTO A VALIDAR — CTRL+ENTER PARA VALIDAR

```
int~suma◁int~a,int~b▷|:return~a+b!:|empty~__main__◁▷|:i
nt~x!int~y!int~res!cin~x!cin~y!res←suma◁x,y▷!cout~res!:|
```

▶ Validar entrada | ✦ Generar ejemplos

TERMINAL DE RESULTADOS — 169PX | Restablecer | Limpiar

Reglas: 105 Validaciones: 4 Correctas: 2 Fallidas: 2 Ejemplos: 0

✓ ✓ VÁLIDO CONTRA
La entrada `int~suma<|int~a,int~b|>|:return~a+b!:|empty~__main__<||>|:int~x!int~y!int~res!cin~x!cin~y!res<-suma<|x,y|>!cout~res!:|` satisface la gramática.

GRAMÁTICA BNF/EBNF — CTRL+ENTER PARA COMPILAR

```
<fin_sentencia>| <break> <fin_sentencia> |
<LecturaEscritura>| <expresion> <fin_sentencia>
<lista_sentencias> ::= <sentencia> <lista_sentencias> |
<comentario> <lista_sentencias> | ε
<LecturaEscritura>::= <cin> <fin_sentencia>| <cout>
<fin_sentencia>
<bloque> ::= <abrir_bloque> <lista_sentencias>
<cerrar_bloque>
<break> ::= "break"
<cin> ::= "cin" <separador> <ID> | "cin" <separador>
<acceder_arreglo>
```

REGLAS DETECTADAS — <programa>

<letra> <texto> <comentario_inicio_linea> <comentario_inicio_bloque> <comentario_fin_bloque> <comentario_linea> <comentario_bloque> <comentario>

TEXTO A VALIDAR — CTRL+ENTER PARA VALIDAR

```
empty~__main__◁▷|:int~arr<<5>>!arr<<0>>←10!arr<<1>>
←20!cout~arr<<0>>!:|
```

▶ Validar entrada | ✦ Generar ejemplos

TERMINAL DE RESULTADOS — 169PX Restablecer Limpiar

Reglas: 105 Validaciones: 5 Correctas: 3 Fallidas: 2 Ejemplos: 0

satisface la gramatica.

✓ ✓ VÁLIDO CONTRA

La entrada `empty~__main__<||>|:int~arr<<5>>!arr<<0>><-10!arr<<1>><-20!cout~arr<<0>>!:|` satisface la gramática.

BNFlab LOCAL — Cargar ejemplo | Limpiar | Compilar gramática

GRAMÁTICA BNF/EBNF — CTRL+ENTER PARA COMPILAR

```
<fin_sentencia>| <break> <fin_sentencia> |
<LecturaEscritura>| <expresion> <fin_sentencia>
<lista_sentencias> ::= <sentencia> <lista_sentencias> |
<comentario> <lista_sentencias> | ε
<LecturaEscritura>::= <cin> <fin_sentencia>| <cout>
<fin_sentencia>
<bloque> ::= <abrir_bloque> <lista_sentencias>
<cerrar_bloque>
<break> ::= "break"
<cin> ::= "cin" <separador> <ID> | "cin" <separador>
<acceder_arreglo>
```

REGLAS DETECTADAS — <programa>

<letra> <texto> <comentario_inicio_linea> <comentario_inicio_bloque> <comentario_fin_bloque> <comentario_linea> <comentario_bloque> <comentario>

TEXTO A VALIDAR — CTRL+ENTER PARA VALIDAR

```
int~multiplicar◁int~a,int~b▷|:return~a*b!:|float~dividir<
|float~a,float~b▷|:return~a/b!:|empty~__main__◁▷|:int~x!
int~y!float~z!cin~x!cin~y!z←dividir◁x,y▷!cout~z!:|
```

▶ Validar entrada | ✦ Generar ejemplos

TERMINAL DE RESULTADOS — 169PX Restablecer Limpiar

Reglas: 105 Validaciones: 8 Correctas: 4 Fallidas: 4 Ejemplos: 0

✓ ✓ VÁLIDO CONTRA

La entrada `int~multiplicar<|int~a,int~b|>|:return~a*b!:|float~dividir<|float~a,float~b|>|:return~a/b!:|empty~__main__<||>|:int~x!int~y!float~z!cin~x!cin~y!z<-dividir<|x,y|>!cout~z!:|` satisface la gramática.

BNFlab LOCAL

Cargar ejemplo | Limpiar | Compilar gramática

GRAMÁTICA BNF/EBNF — CTRL+ENTER PARA COMPILAR

```
<fin_sentencia>| <break> <fin_sentencia> |
<LecturaEscritura>| <expresion> <fin_sentencia>
<lista_sentencias> ::= <sentencia> <lista_sentencias> |
<comentario> <lista_sentencias> | ε
<LecturaEscritura>::= <cin> <fin_sentencia>| <cout>
<fin_sentencia>
<bloque> ::= <abrir_bloque> <lista_sentencias>
<cerrar_bloque>
<break> ::= "break"
<cin> ::= "cin" <separador> <ID> | "cin" <separador>
<acceder_arreglo>
```

REGLAS DETECTADAS — <programa>

<letra> <texto> <comentario_inicio_linea> <comentario_inicio_bloque> <comentario_fin_bloque> <comentario_linea> <comentario_bloque> <comentario>

TEXTO A VALIDAR — CTRL+ENTER PARA VALIDAR

```
empty~__main__◁ ▷|:int~contador!contador←0!++contador!cou
t~contador!--contador!:|
```

▶ Validar entrada | ✦ Generar ejemplos

TERMINAL DE RESULTADOS — 169PX | Restablecer | Limpiar

Reglas: 105 Validaciones: 9 Correctas: 5 Fallidas: 4 Ejemplos: 0

nt~y!float~z!cin~x!cin~y!z<-dividir<|x,y|>!cout~z!:| satisface la gramática.

✓ ✓ VÁLIDO CONTRA

La entrada empty~__main__<||>|:int~contador!contador<-0!++contador!cout~contador!--contador!:| satisface la gramática.